

RoadSense — technical report

Team Feba: Shuxratov Shoxijaxonbek (Team captain · project manager), Ibragimov Diyorbek (Frontend developer), Lutfullayev Mirfayz (Backend developer)

WIUT Hackathon 2026, Computer Vision track · code <https://github.com/Shoxijaxonbek/Feba> · website and live demo shoxijaxonbek-feba.static hf.space

0.86

Score A on our own labels
of C3902

101

events in 18.4 min of 4K
video (4 samples)

1.8×

video length for Part A +
B (limit 3×)

0

false accident alarms on
the samples

What we built

RoadSense turns the fixed 4K view of a signalised Tashkent junction into a list of traffic events `[start_sec, end_sec, label]` (Part A) and a causal, frame-by-frame accident-risk score (Part B). It is a little learning and a lot of explicit geometry:

1. **Decode only what we use.** The camera writes H.264 High 4:2:2 10-bit at ~150 Mbit/s. PyAV skips non-reference frames: the I-B-B-P GOP yields every third frame (~10 fps) at ~5× realtime.
2. **Detect and track.** YOLO11m (COCO weights, not fine-tuned) with pre- and post-processing on the GPU; ByteTrack separately for vehicles, two-wheelers, pedestrians and possible obstacles.
3. **Know the scene.** Carriageway, islands, three zebras, the stop line, queue zones, the junction box and the bus stop are drawn once on a reference view. Every video is registered to it with SIFT features, because the tripod moved by up to 80 px between the four recordings.
4. **Read the signal.** Each lamp is located as the spot whose colour switches on and off over the video, then read frame by frame. That works in harsh midday sun and at dusk.
5. **One rule per class**, written from the annotation conventions (start and end of each class).
6. **Part B:** causal tracking at 5 fps, the time until two ground footprints would touch, the deceleration needed to avoid it, a conflict that must hold for ~0.6 s, and a calibration that keeps normal traffic below the alarm level.

What worked

- **Decoding only reference frames** made Part A take about half the video's length with no visible loss: 10 fps is plenty for tracking at junction speeds.
- **Reviewing every detection by eye.** We rendered each event as an image strip and looked at all of them. Almost every false positive had a reason we could turn into a rule condition, not a threshold tweak: people waiting at zebra ends, parked cars, passengers seen through bus windows, boxes cut off at the frame edge, ID switches between buses, left-turners and queues at other approaches waiting inside the junction, a bonnet over the stop line.
- **Telling queues from stopped vehicles by their neighbours.** A queue stands still together; a stopped vehicle stands still while the traffic around it keeps moving.
- **Predicting only the classes we can detect.** Score A averages over every class that appears in the ground truth *or* the predictions, so a class we guess wrongly adds a zero.

What did not work

- **The first versions did not survive new footage.** Everything was first tuned on the one dusk video we could download. The midday videos broke the registration (ORB), the signal reader (a lit lamp in direct

sun is barely brighter than its housing), wrong-way (cars from the other approach turn through the junction) and stopped_vehicle (queues everywhere). Each fix is listed under failure cases.

- **Part B** raised false alarms three times: first from centre-to-centre time-to-collision, then from a preprocessing mismatch with Part A, then from one-frame spikes of cars merging at midday. We still have **no accident or near miss** to calibrate on, so the alarm level is set from normal traffic only.
- **failure_to_yield** is our noisiest class: we cannot yet tell a pedestrian about to step off the curb from one who is waiting.
- **Illegal or legal?** Cars regularly U-turn around the median end at midday. We do not know whether the annotators treat these as illegal; we report them.
- **Our own labels** of the three videos we got last were made by reviewing our detections, so they measure precision, not recall. Only C3902's labels were built from loose-threshold candidates.

What we would do next

- Fine-tune the detector on a few hundred frames of this camera (small far-side pedestrians and scooters, which COCO does not have).
- Use pedestrian motion (heading onto the road vs. standing) for failure_to_yield.
- Collect or stage accident and near-miss clips to fit the Part B calibration on real positives.
- Trace the lane markings on an empty-road frame to add solid_line_crossing and illegal_turn.

Per-class results on the dev labels (C3902)

class	F1@0.3	F1@0.5	F1@0.7	TP/FP/FN @0.5
failure_to_yield	0.813	0.625	0.562	10/9/3
illegal_u_turn	1.0	1.0	1.0	1/0/0
jaywalking	0.769	0.769	0.769	5/2/1
stopped_vehicle	1.0	1.0	1.0	1/0/0

Dev sets (submitted pipeline, predictions_samples.json)

Variant	Score A	micro F1@0.5
C3902 (labels from loose-threshold candidates, reviewed)	0.859	0.694
C3896 + C3897 + C3905 (our detections, reviewed: precision only)	0.998	0.993
All four samples	0.97	0.918

Detector size and sampling rate (Part A, dev labels)

Variant	Score A	micro F1@0.5	perception x realtime
YOLO11m @ 10 fps (submitted)	0.859	0.694	0.35
YOLO11s @ 10 fps	0.65	0.604	0.29
YOLO11m @ 5 fps	0.569	0.6	0.24
YOLO11s @ 5 fps	0.321	0.549	0.24

Runtime of the official harness (RTX 4050 laptop GPU)

video	length s	Part A s	Part B s	limit s	x video
C3896.MP4	340.3	174.4	507.9	1021.0	2.0
C3897.MP4	317.8	140.1	415.3	953.5	1.75
C3902.MP4	317.8	149.2	414.4	953.5	1.77
C3905.MP4	127.6	58.4	155.6	382.9	1.68